

សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប

មហាវិទ្យាល័យ វិទ្យាសាស្ត្រ និងបច្ចេកវិទ្យា



មុខវិជ្ជា៖

Programming Methodology in C++
មេរៀនទី៧៖

ជំពូក ៧ 5. ការប្រើប្រាស់ Overloading the
Operators, Static Member

ឆ្នាំទី ១ ឆមាស ទី ២

គណៈកម្មការអភិវឌ្ឍន៍កម្មវិធីសិក្សា

© 2024

មតិកា

- ❑ 1. ការបង្កើត និង ប្រើប្រាស់ Classes និង Objects
- ❑ 2. ការប្រើប្រាស់ Constructor និង Destructor
- ❑ 3. ការប្រើប្រាស់ Pointer to Class
- ❑ 4. ការប្រើប្រាស់ Class Defined with Structure , Union
- ❑ 5. ការប្រើប្រាស់ Overloading the Operators, Static Member
- ❑ 6. ការប្រើប្រាស់ FriendShip និង Inheritance
- ❑ 7. ការប្រើប្រាស់ Single និង Multiple Inheritance
- ❑ 8. ការប្រើប្រាស់ Pointer to Base Class Abstract Base Class

មតិកា

- ❑9. ការប្រើប្រាស់ Function Template និង Class Template
- ❑10. ការប្រើប្រាស់ Virtual Member , Template Specialization
- ❑11. ការប្រើប្រាស់ NameSpace និង ដោះស្រាយ Exception
- ❑ 12. ការប្រើ ការបំបែកប្រភេទទិន្នន័យ
- ❑១៣. ការកំណត់ Preprocessor Directives ការប្រើ Input/ output with Files.
- ❑14. ការប្រតិបត្តិរបៀប Read និង Write Files

5. ការប្រើប្រាស់ Overloading the Operators, Static Member

- 1.១. អ្វីទៅ ហៅថា Overloading the Operators?
- ១.២. ការប្រើប្រាស់ Overloading the Operators.
- ១.3. អ្វីទៅ ហៅ ថា Static Member?
- 1.4. ការប្រើប្រាស់ Static Member.
- Example : Coding .

1.១. អ្វីទៅ ហៅថា Overloading the Operators?

- **Overloading the Operators** នៅក្នុង C++ គឺជាបច្ចេកទេសដែលអនុញ្ញាតឱ្យអ្នកប្តូររូបរាងនៃ កំណត់ន័យ (behavior) នៃ operators (ដូចជា +, -, *, =) សម្រាប់ប្រភេទទិន្នន័យផ្ទាល់ខ្លួន (user-defined data types) ឬ objects ។ វាអនុញ្ញាតឱ្យអ្នកធ្វើឱ្យ operators ទំនងនឹងគោល ន័យដើម ប៉ុន្តែមានលក្ខណៈបន្ថែមជាមួយលក្ខណៈពិសេសដែលអ្នកបានកំណត់។

១.២. ការប្រើប្រាស់ Overloading the Operators

❑ លក្ខណៈពិសេសនៃ Operator Overloading

➤ **ពង្រីកបំណែកគោលនៃ Operators:** អ្នកអាចប្រើ operator ជាមួយ objects

ដូចដែលអ្នកប្រើជាមួយ primitive types ដូចជា int, float។ **បង្កើតសមត្ថភាពថ្មីៗ**

សម្រាប់ Objects: អ្នកអាចសរសេរកូដដែលមានភាពច្បាស់ និងងាយស្រួលក្នុង

ការយល់ ដោយធ្វើឱ្យ operators អាចដំណើរការជាមួយ objects នៃ classes ផ្ទាល់ខ្លួន

នេះអាចប្រើជាមួយ **Function ឬ Method:** Operator overloading ត្រូវបានសរសេរ

ជា function ឬ method ដែលធ្វើការបញ្ចូលវត្ថុ objects និងសងវិញតម្លៃចេញ។

3. វិធីសាស្ត្រដ៏សំខាន់នៃ Operator Overloading

- **Member Function Overloading:** ការប្រើប្រាស់ operators អាចត្រូវបានសរសេរជា member functions ដែលធ្វើការបញ្ចូល objects។
- **Non-Member Function Overloading:** អ្នកអាចសរសេរ operator overloading ជា non-member functions ដែលអាចប្តូរគោលន័យតាមតម្រូវការ។
- **Restrictions:** មួយចំនួននៃ operators មិនអាចត្រូវបាន overload ឬប្តូរគោលន័យ ដូចជា ::, .*, .:, sizeof, ?: ។

ប្រយោជន៍នៃ Operator Overloading

- **បន្ថែមអានក្នុងច្បាប់លាស់ និងងាយស្រួល:** Operator overloading ផ្តល់នូវវិធីសាស្ត្រសម្រាប់ការដំណើរការដែលមានគោលន័យជាមួយ objects ដោយប្រើ operators ដែលស្រដៀងនឹង operators ដែលអ្នកស្គាល់។ **បន្ថែមសមត្ថភាពសម្រាប់ Classes:** Operator overloading ផ្តល់សមត្ថភាពសម្រាប់ការគ្រប់គ្រង និងដំណើរការវត្ថុ objects ដែលមានលក្ខណៈស្រដៀងនឹង primitive types។

Coding Operator Overloading

```

1 #include <iostream>
2 using namespace std;
3
4 class Complex {
5 public:
6     int real, imag;
7
8     // Constructor
9     Complex(int r = 0, int i = 0) {
10         real = r;
11         imag = i;
12     }
13
14     // Operator Overloading for "+"
15     Complex operator + (const Complex &obj) {
16         Complex result;
17         result.real = real + obj.real;
18         result.imag = imag + obj.imag;
19         return result;
20     }
21 };
22
23 int main() {
24     Complex c1(10, 5), c2(2, 4);
25
26     // Overloaded "+" operator used
27     Complex c3 = c1 + c2;
28
29     cout << "c3.real = " << c3.real << endl;
30     cout << "c3.imag = " << c3.imag << endl;
31
32     return 0;
33 }
34

```

STDIN

Input for the program (Optional)

Output:

```

c3.real = 12
c3.imag = 9

```

១.3. អ្វីទៅ ហៅ ថា Static Member?

- **Static Member** នៅក្នុង C++ គឺជាសមាជិកនៃ class ដែលត្រូវបានចែងច្បាស់ជាមនុស្សជាថា "static"។ វាអាចជាអថេរ (variable) ឬ method (function) ដែលមានលក្ខណៈពិសេសមួយចំនួនដូចខាងក្រោម:

លក្ខណៈពិសេសនៃ Static Member

- **បែងចែកឯកជនទិន្នន័យជាសកល (Shared among all objects):**

- Static member variables ត្រូវបានចែករំលែករវាង objects ទាំងអស់នៃ class ។ វាមានទីតាំង memory តែមួយគត់ ហើយតម្លៃរបស់វាត្រូវបានចែករំលែកចំពោះ objects ទាំងអស់នៃ class នោះ។ វាមិនត្រូវបានក្លាយទៅជាអថេរផ្សេងៗសម្រាប់ objects ដូចជា non-static member variables។

- **អាចចូលប្រើដោយមិនចាំបាច់មាន object (Accessed without an Object):**

- Static members អាចត្រូវបានចូលប្រើដោយប្រើឈ្មោះ class របស់វាផ្ទាល់ ដោយគ្មានការបង្កើត object ឬ instance នៃ class ។ វាអាចត្រូវបានចូលប្រើដោយគ្រាន់តែប្រើ `ClassName::staticMemberName`។

- **ពិភាក្សាពេលកំណត់តម្លៃ (Initialized only once):**

- Static member variables ត្រូវបានកំណត់តម្លៃតែម្តងនៅពេលត្រូវបានដាក់ដំណើរការដំបូងគេ ហើយតម្លៃនោះនឹងនៅតែជាការស្ថិតស្ថេរនៅគ្រប់ពេលដែលអាចដំណើរការជាមួយ objects។

- **Static member functions (Method) អាចប្រើតែ static members:**

- Static member functions អាចចូលប្រើ static members នៃ class ប៉ុណ្ណោះ ហើយមិនអាចប្រើ non-static members ទេ ដោយសារតែ static methods មិនមាន access to the instance (i.e., this pointer) នៃ object។

ប្រយោជន៍នៃ Static Members

- **ចែករំលែកទិន្នន័យរវាង objects:** ជាមួយ static members, អ្នកអាចចែករំលែកទិន្នន័យមួយចំនួនរវាង objects ទាំងអស់នៃ class ដោយគ្មានការបង្កើត copy របស់ទិន្នន័យនោះសម្រាប់ objects នីមួយៗ។
- **អាចប្រើជាមួយ class-level functions:** Static member functions អាចប្រើសម្រាប់ការគ្រប់គ្រងការងារជាកម្រិត class-level ដោយមិនចាំបាច់មាន object ។ វាមានប្រយោជន៍សម្រាប់ការងារដែលត្រូវបានអនុវត្តនៅលើ class ដោយសារតែវាគ្មានតម្រូវការទៅកាន់ object ដូចជា non-static functions។
- **សមត្ថភាពខ្ពស់ក្នុងការគ្រប់គ្រងពេលវេលា និងទិន្នន័យសកល:** វាមានសក្តានុពលសម្រាប់ការគ្រប់គ្រងធនធាន និងការងារដែលមានតម្រូវការប្រើប្រាស់ទិន្នន័យសកលចំពោះ objects ។

Coding Static Members

```
#include<iostream>
using namespace std;

class MyClass {
public:
    static int staticVar; // Static member variable

    // Static member function
    static void staticFunc() {
        cout << "Static variable value: " << staticVar << endl;
    }
};

// Initialize static member variable outside the class
int MyClass::staticVar = 0;

int main() {
    MyClass obj1, obj2;

    // Access static member using class name
    MyClass::staticVar = 10;

    // Access static member function using class name
    MyClass::staticFunc();

    // Access static member using object (not recommended)
    obj1.staticVar = 20;
    obj2.staticFunc();

    return 0;
}
```

STDIN

Input for the program (Optional)

Output:

Static variable value: 10

Static variable value: 20



សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប

ASIA EURO UNIVERSITY

សិក្សាស្រាវជ្រាវ គ្រប់ទីកន្លែង

អរគុណ

សម្រាប់ការយកចិត្តទុកដាក់

<https://elearning.aeu.cloud>